Міністерство освіти і науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни «Об’єктно-орієнтоване програмування»

на тему:

«Вдосконалення структури коду графічного редактора об’єктів»

Виконав:
студент групи ІМ-55
Присяжний С. О.
Номер у списку: 25
(Варіант Ж = 25)

Перевірив:
Рекечинський Д. О.

Київ 2026

Мета роботи:

Отримати стійкі практичні вміння та навички проектування та рефакторингу класів у стилі об'єктно-орієнтованого програмування. Усунути проблему небажаного зростання залежностей і жорсткої зв'язаності коду шляхом дотримання принципу відкритості/закритості (Open/Closed Principle — OCP) та впровадження залежностей (Dependency Injection). Забезпечити зручне розширення функціоналу системи шляхом додавання нових типів складних об'єктів на основі механізму множинного успадкування (Multiple Inheritance).

Завдання:

1. Провести архітектурний рефакторинг головного класу-координатора графічного редактора, реорганізувавши його в клас MyEditor, незалежний від списку конкретних редакторів.

2. Реалізувати універсальний метод запуску редагування start(editor, mode, modeName) без використання окремих фіксованих методів типу startPointEditor(), startLineEditor() тощо.

3. Відповідно до індивідуального непарного варіанта Ж = 25 забезпечити функціонування статичного глобального об'єкта класу MyEditor (патерн Singleton).

4. Реалізувати дві нові складні геометричні форми на базі множинного успадкування:
   • LineOOShape (лінія з кружечками на кінцях) — успадкування від LineShape та EllipseShape;
   • CubeShape (просторовий каркас куба) — успадкування від RectShape та LineShape.

5. Забезпечити виклик методів show() відповідних базових класів у методах show() похідних форм LineOOShape та CubeShape без заміни множинного успадкування композицією.

6. Згідно з методичними вимогами налаштувати відображення «гумового» сліду при інтерактивному введенні усіх шести фігур пунктирною лінією.

7. Розширити панель інструментів (Toolbar) та системне меню «Об'єкти» до 6 кнопок із підказками (Tooltips) та радіо-індикацією активного інструмента.

8. Побудувати детальну UML-діаграму класів із демонстрацією множинного успадкування та ромбовидної ієрархії, навести скріншоти тестування, оформити висновки та надати відповіді на контрольні запитання.

Розрахунок та характеристика індивідуального варіанта:

Відповідно до п. 2 варіантів завдань лабораторної роботи №4, номер варіанта дорівнює номеру студента зі списку в академічному журналі: Ж = 25 (непарний номер).

Характеристики індивідуального варіанта Ж = 25:

1. Організація головного об'єкта класу MyEditor:

Умова вибору: студенти з непарним номером (1, 3, 5, ..., 25) програмують глобальний статичний об'єкт класу MyEditor.
Реалізація: клас MyEditor спроєктовано за патерном Singleton із глобальним статичним методом доступу getInstance() та приватним конструктором.

2. Буфер графічних примітивів Shape:

Масив збереження фігур pcshape організовано як фіксований статичний масив розміром N = 126 покажчиків із інкапсульованим лічильником shapeCount.

3. Стиль «гумового» сліду при введенні об'єктів:

Згідно з вимогою №3 до Лаб №4, гумовий слід при інтерактивному введенні усіх шести типів геометричних фігур малюється пунктирною лінією (ctx.setLineDash([6, 6])) синього кольору (#2563eb).

4. Кольори та стилі раніше створених геометричних форм:

Усі кольори та параметри відображення базових чотирьох фігур зберігаються аналогічно лабораторній роботі №3:
• Крапка: чорний колір;
• Лінія: чорний контур;
• Прямокутник: введення по двох кутах, чорний контур, блакитне заповнення (#38bdf8);
• Еліпс: введення від центру до кута охоплюючого прямокутника, чорний контур, біле заповнення (#ffffff).

5. Нові геометричні форми на базі множинного успадкування:

• Лінія з кружечками (LineOOShape): множинне успадкування від LineShape та EllipseShape. Малює лінію та 2 кружечки на її кінцях.
• Каркас куба (CubeShape): множинне успадкування від RectShape та LineShape. Малює просторову конструкцію з передньої та зміщеної задньої граней із сполучними ребрами.

6. Інтерфейс користувача (Toolbar та Меню):

Панель інструментів розширено до 6 кнопок із векторними піктограмами, спливаючими підказками (Tooltips) та фіксацією стану. Меню «Об'єкти» містить 6 підпунктів із радіо-позначкою вибору та гарячими клавішами Ctrl+1...Ctrl+6.

Схеми та діаграми структури програми:

На основі вдосконаленої об'єктно-орієнтованої структури розроблено повну UML-діаграму класів (Рис. 1) з чітким відображенням множинного успадкування для класів LineOOShape та CubeShape, а також схему архітектури компонентів та IPC-взаємодії (Рис. 2).

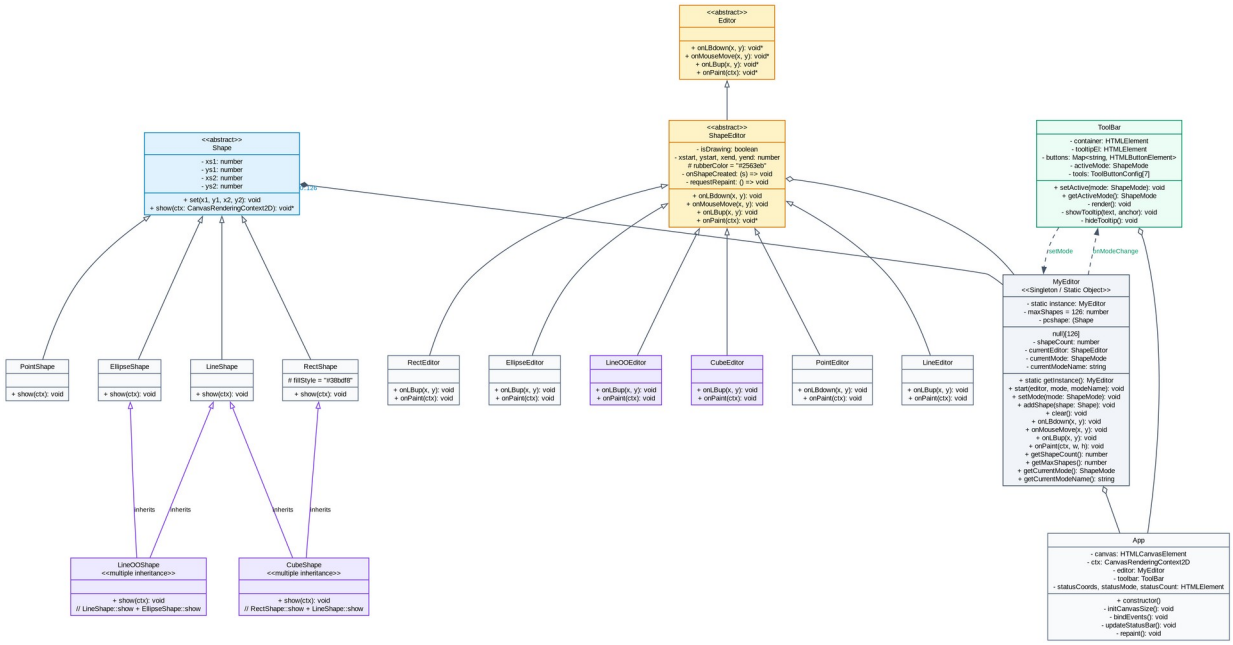


Рис. 1. UML діаграма класів графічного редактора (ієрархія фігур із множинним успадкуванням LineOOShape та CubeShape, ієрархія редакторів, координатор MyEditor та компонент ToolBar)

Аналіз діаграми класів (Рис. 1):

1. Базова ієрархія Shape: абстрактний клас Shape визначає спільні поля прямокутних координат (xs1, ys1, xs2, ys2) та віртуальний метод show(ctx). Похідні базові класи PointShape, LineShape, RectShape та EllipseShape перевизначають його.

2. Множинне успадкування нових фігур:

• LineOOShape одночасно успадковує функціональність LineShape та EllipseShape. Метод show() викликає LineShape::show для відмальовки центральної осі та EllipseShape::show для генерації кінцевих кілець;

• CubeShape одночасно успадковує функціональність RectShape та LineShape. Метод show() викликає RectShape::show для передньої та задньої граней і LineShape::show для чотирьох просторових ребер.

3. Відсутність жорстких зв'язків у MyEditor (Open/Closed Principle): координатор MyEditor більше не містить методів, прив'язаних до конкретних типів фігур (startPointEditor тощо). Замість цього використовується узагальнений метод start(editor, mode, modeName). Завдяки цьому додавання LineOO та Cube відбулося без модифікації базового контракту координатора.

4. Компонент ToolBar інкапсулює 6 кнопок швидкого вибору, векторні піктограми, стан натискання (pressed state) та спливаючі підказки.

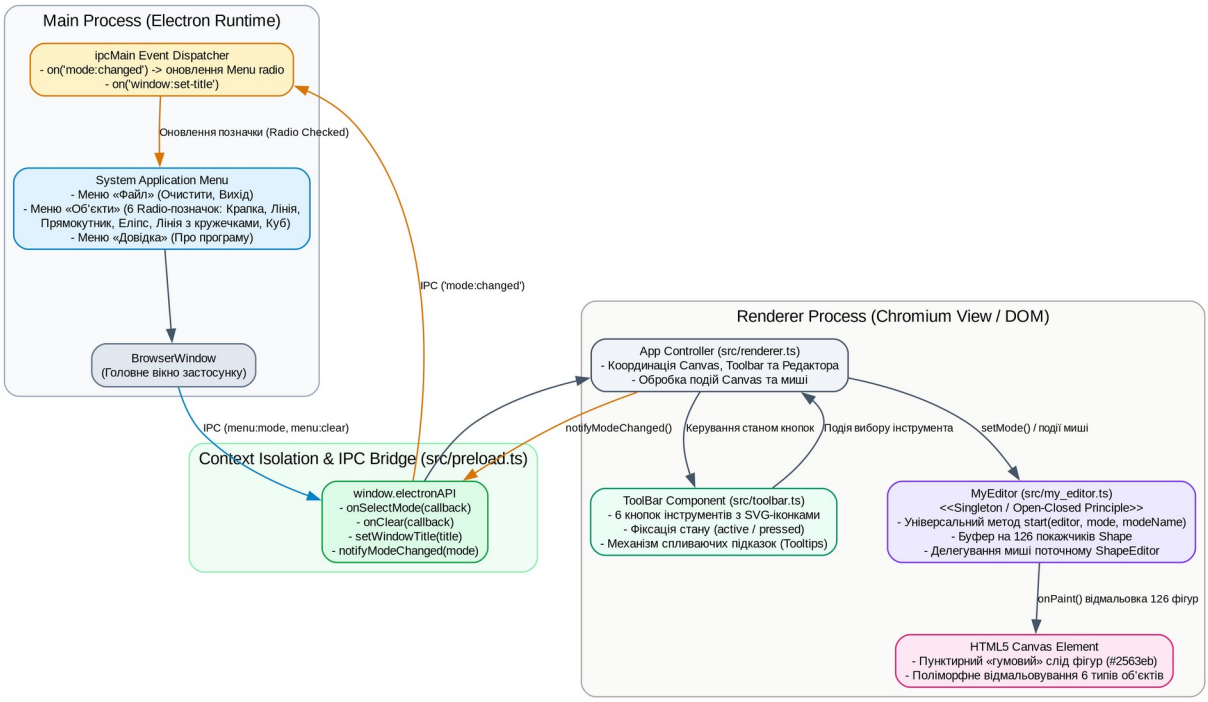


Рис. 2. Архітектура модулів та схема IPC-взаємодії системного меню, панелі інструментів, координатора MyEditor та полотна Canvas

Діаграма архітектури (Рис. 2) ілюструє розділення відповідальності між головним системним процесом Electron Main (нативне вікно, 6-позиційне радіо-меню «Об'єкти»), захищеним мостом Preload Bridge та Renderer-процесом, де координуються полотно, панель інструментів та глобальний статичний об'єкт MyEditor.

Вихідний текст програми:

Нижче наведено вихідні тексти ключових модулів модернізованого графічного редактора. Особливу увагу приділено класам множинного успадкування LineOOShape та CubeShape, новому класу координатора MyEditor та редакторам введення нових об'єктів.

src/my_editor.ts (Головний клас-координатор MyEditor: принцип OCP, Singleton, буфер 126 фігур)

```
import { Shape } from './shapes/shape.ts';
import { ShapeEditor } from './editors/shape_editor.ts';
import { PointEditor } from './editors/point_editor.ts';
import { LineEditor } from './editors/line_editor.ts';
import { RectEditor } from './editors/rect_editor.ts';
import { EllipseEditor } from './editors/ellipse_editor.ts';
import { LineOOEditor } from './editors/line_oo_editor.ts';
import { CubeEditor } from './editors/cube_editor.ts';

export type ShapeMode =
  'point' | 'line' | 'rect' | 'ellipse' | 'lineOO' | 'cube';

export class MyEditor {
  private static instance: MyEditor | null = null;
```

```
private readonly maxShapes: number = 126;
private pcshape: (Shape | null)[] = Array.from({ length: 126 }, () => null);
private shapeCount: number = 0;

private currentEditor: ShapeEditor | null = null;
private currentMode: ShapeMode = 'point';
private currentModeName: string = 'Крапка';

constructor(
  private readonly requestRepaint: () => void,
  private readonly onTitleChange: (title: string) => void,
  private readonly onModeChange?: (mode: ShapeMode) => void,
) {
  this.setMode('point');
}

public static getInstance(
  requestRepaint?: () => void,
  onTitleChange?: (title: string) => void,
  onModeChange?: (mode: ShapeMode) => void,
): MyEditor {
  if (!MyEditor.instance) {
    if (!requestRepaint || !onTitleChange) {
      throw new Error('MyEditor must be initialized with callbacks');
    }
    MyEditor.instance = new MyEditor(
      requestRepaint,
      onTitleChange,
      onModeChange,
    );
  }
  return MyEditor.instance;
}

addShape(shape: Shape): void {
  if (this.shapeCount < this.maxShapes) {
    this.pcshape[this.shapeCount++] = shape;
  }
}

start(editor: ShapeEditor, mode: ShapeMode, modeName: string): void {
  this.currentEditor = editor;
  this.currentMode = mode;
  this.currentModeName = modeName;
  this.updateTitle();
  this.onModeChange?.(this.currentMode);
}
```

```
setMode(mode: ShapeMode): void {
  const notifyShape = (s: Shape): void => this.addShape(s);
  const repaint = (): void => this.requestRepaint();

  switch (mode) {
    case 'point':
      this.start(new PointEditor(notifyShape, repaint), 'point', 'Крапка');
      break;
    case 'line':
      this.start(new LineEditor(notifyShape, repaint), 'line', 'Лінія');
      break;
    case 'rect':
      this.start(new RectEditor(notifyShape, repaint), 'rect', 'Прямокутник');
      break;
    case 'ellipse':
      this.start(new EllipseEditor(notifyShape, repaint), 'ellipse', 'Еліпс');
      break;
    case 'lineOO':
      this.start(
        new LineOOEditor(notifyShape, repaint),
        'lineOO',
        'Лінія з кружечками',
      );
      break;
    case 'cube':
      this.start(new CubeEditor(notifyShape, repaint), 'cube', 'Каркас куба');
      break;
  }
}

private updateTitle(): void {
  this.onTitleChange(
    `Графічний редактор — Лабораторна робота №4 (${this.shapeCount}/${this.maxShapes})`,
  );
}

clear(): void {
  this.pcshape.fill(null);
  this.shapeCount = 0;
  this.updateTitle();
  this.requestRepaint();
}

onLBdown(x: number, y: number): void {
  this.currentEditor?.onLBdown(x, y);
}

onMouseMove(x: number, y: number): void {
  this.currentEditor?.onMouseMove(x, y);
```

```
  }

  onLBup(x: number, y: number): void {
    this.currentEditor?.onLBup(x, y);
    this.updateTitle();
  }

  onPaint(ctx: CanvasRenderingContext2D, width: number, height: number): void {
    ctx.fillStyle = '#ffffff';
    ctx.fillRect(0, 0, width, height);

    for (let i = 0; i < this.shapeCount; i++) {
      this.pcshape[i]?.show(ctx);
    }

    this.currentEditor?.onPaint(ctx);
  }

  getShapeCount(): number {
    return this.shapeCount;
  }

  getMaxShapes(): number {
    return this.maxShapes;
  }

  getCurrentMode(): ShapeMode {
    return this.currentMode;
  }

  getCurrentModeName(): string {
    return this.currentModeName;
  }
}
```

src/shapes/line_oo_shape.ts (Клас LineOOShape: множинне успадкування від LineShape та EllipseShape)

```
import { LineShape } from './line_shape.ts';
import { EllipseShape } from './ellipse_shape.ts';

export class LineOOShape extends LineShape {
  override show(ctx: CanvasRenderingContext2D): void {
    const startX = this.xs1;
    const startY = this.ys1;
    const endX = this.xs2;
    const endY = this.ys2;

    super.show(ctx);
```

```
    const radius = 8;

    this.xs1 = startX;
    this.ys1 = startY;
    this.xs2 = startX + radius;
    this.ys2 = startY + radius;
    EllipseShape.prototype.show.call(this, ctx);

    this.xs1 = endX;
    this.ys1 = endY;
    this.xs2 = endX + radius;
    this.ys2 = endY + radius;
    EllipseShape.prototype.show.call(this, ctx);

    this.xs1 = startX;
    this.ys1 = startY;
    this.xs2 = endX;
    this.ys2 = endY;
  }
}
```

src/shapes/cube_shape.ts (Клас CubeShape: множинне успадкування від RectShape та LineShape)

```
import { RectShape } from './rect_shape.ts';
import { LineShape } from './line_shape.ts';

export class CubeShape extends RectShape {
  constructor(x1: number = 0, y1: number = 0, x2: number = 0, y2: number = 0) {
    super(x1, y1, x2, y2);
    this.fillStyle = 'rgba(56, 189, 248, 0.2)';
  }

  override show(ctx: CanvasRenderingContext2D): void {
    const origX1 = this.xs1;
    const origY1 = this.ys1;
    const origX2 = this.xs2;
    const origY2 = this.ys2;

    const left = Math.min(origX1, origX2);
    const top = Math.min(origY1, origY2);
    const right = Math.max(origX1, origX2);
    const bottom = Math.max(origY1, origY2);

    const width = right - left;
    const height = bottom - top;
    const ox = Math.round(width * 0.35);
    const oy = Math.round(height * 0.35);
```

```
    this.xs1 = left + ox;
    this.ys1 = top - oy;
    this.xs2 = right + ox;
    this.ys2 = bottom - oy;
    super.show(ctx);

    const drawEdge = (x1: number, y1: number, x2: number, y2: number): void => {
      this.xs1 = x1;
      this.ys1 = y1;
      this.xs2 = x2;
      this.ys2 = y2;
      LineShape.prototype.show.call(this, ctx);
    };

    drawEdge(left, top, left + ox, top - oy);
    drawEdge(right, top, right + ox, top - oy);
    drawEdge(left, bottom, left + ox, bottom - oy);
    drawEdge(right, bottom, right + ox, bottom - oy);

    this.xs1 = left;
    this.ys1 = top;
    this.xs2 = right;
    this.ys2 = bottom;
    super.show(ctx);

    this.xs1 = origX1;
    this.ys1 = origY1;
    this.xs2 = origX2;
    this.ys2 = origY2;
  }
}
```

src/editors/line_oo_editor.ts (Контролер введення лінії з кружечками з пунктирним слідом)

```
import { ShapeEditor } from './shape_editor.ts';
import { LineOOShape } from '../shapes/line_oo_shape.ts';

export class LineOOEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new LineOOShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const radius = 8;
```

```
    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.setLineDash([6, 6]);

    ctx.beginPath();
    ctx.moveTo(this.xstart, this.ystart);
    ctx.lineTo(this.xend, this.yend);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(this.xstart, this.ystart, radius, 0, Math.PI * 2);
    ctx.stroke();

    ctx.beginPath();
    ctx.arc(this.xend, this.yend, radius, 0, Math.PI * 2);
    ctx.stroke();

    ctx.restore();
  }
}
```

src/editors/cube_editor.ts (Контролер введення каркаса куба з об'ємним пунктирним слідом)

```
import { ShapeEditor } from './shape_editor.ts';
import { CubeShape } from '../shapes/cube_shape.ts';

export class CubeEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new CubeShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const left = Math.min(this.xstart, this.xend);
    const top = Math.min(this.ystart, this.yend);
    const right = Math.max(this.xstart, this.xend);
    const bottom = Math.max(this.ystart, this.yend);

    const width = right - left;
    const height = bottom - top;
    const ox = Math.round(width * 0.35);
    const oy = Math.round(height * 0.35);
```

```
    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.setLineDash([6, 6]);

    ctx.strokeRect(left, top, width, height);
    ctx.strokeRect(left + ox, top - oy, width, height);

    ctx.beginPath();
    ctx.moveTo(left, top);
    ctx.lineTo(left + ox, top - oy);
    ctx.moveTo(right, top);
    ctx.lineTo(right + ox, top - oy);
    ctx.moveTo(left, bottom);
    ctx.lineTo(left + ox, bottom - oy);
    ctx.moveTo(right, bottom);
    ctx.lineTo(right + ox, bottom - oy);
    ctx.stroke();

    ctx.restore();
  }
}
```

src/shapes/rect_shape.ts (Клас прямокутника з підтримкою налаштовуваної заливки fillStyle)

```
import { Shape } from './shape.ts';

export class RectShape extends Shape {
  protected fillStyle: string = '#38bdf8';

  show(ctx: CanvasRenderingContext2D): void {
    const left = Math.min(this.xs1, this.xs2);
    const top = Math.min(this.ys1, this.ys2);
    const width = Math.abs(this.xs2 - this.xs1);
    const height = Math.abs(this.ys2 - this.ys1);

    ctx.save();
    if (this.fillStyle && this.fillStyle !== 'transparent') {
      ctx.fillStyle = this.fillStyle;
      ctx.fillRect(left, top, width, height);
    }
    ctx.strokeStyle = '#000000';
    ctx.lineWidth = 1;
    ctx.strokeRect(left, top, width, height);
    ctx.restore();
  }
}
```

src/editors/shape_editor.ts (Базовий клас редакторів введення з синім кольором сліду)

```
import { Editor } from './editor.ts';
import { Shape } from '../shapes/shape.ts';

export abstract class ShapeEditor extends Editor {
  protected isDrawing: boolean = false;
  protected xstart: number = 0;
  protected ystart: number = 0;
  protected xend: number = 0;
  protected yend: number = 0;

  protected readonly rubberColor: string = '#2563eb';

  constructor(
    protected readonly onShapeCreated: (shape: Shape) => void,
    protected readonly requestRepaint: () => void,
  ) {
    super();
  }

  onLBdown(x: number, y: number): void {
    this.xstart = this.xend = x;
    this.ystart = this.yend = y;
    this.isDrawing = true;
    this.requestRepaint();
  }

  onMouseMove(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.requestRepaint();
  }

  onLBup(x: number, y: number): void {
    if (!this.isDrawing) return;
    this.xend = x;
    this.yend = y;
    this.isDrawing = false;
  }
}
```

src/editors/line_editor.ts (Редактор відрізків із пунктирним синім слідом [6, 6])

```
import { ShapeEditor } from './shape_editor.ts';
import { LineShape } from '../shapes/line_shape.ts';

export class LineEditor extends ShapeEditor {
```

```
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new LineShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.setLineDash([6, 6]);
    ctx.beginPath();
    ctx.moveTo(this.xstart, this.ystart);
    ctx.lineTo(this.xend, this.yend);
    ctx.stroke();
    ctx.restore();
  }
}
```

src/editors/rect_editor.ts (Редактор прямокутників із пунктирним синім слідом [6, 6])

```
import { ShapeEditor } from './shape_editor.ts';
import { RectShape } from '../shapes/rect_shape.ts';

export class RectEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new RectShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const left = Math.min(this.xstart, this.xend);
    const top = Math.min(this.ystart, this.yend);
    const width = Math.abs(this.xend - this.xstart);
    const height = Math.abs(this.yend - this.ystart);

    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.setLineDash([6, 6]);
    ctx.beginPath();
    ctx.strokeRect(left, top, width, height);
```

```
    ctx.restore();
  }
}
```

src/editors/ellipse_editor.ts (Редактор еліпсів із пунктирним синім слідом [6, 6])

```
import { ShapeEditor } from './shape_editor.ts';
import { EllipseShape } from '../shapes/ellipse_shape.ts';

export class EllipseEditor extends ShapeEditor {
  override onLBup(x: number, y: number): void {
    super.onLBup(x, y);
    this.onShapeCreated(
      new EllipseShape(this.xstart, this.ystart, this.xend, this.yend),
    );
    this.requestRepaint();
  }

  override onPaint(ctx: CanvasRenderingContext2D): void {
    if (!this.isDrawing) return;
    const rx = Math.abs(this.xend - this.xstart);
    const ry = Math.abs(this.yend - this.ystart);

    if (rx === 0 && ry === 0) return;

    ctx.save();
    ctx.strokeStyle = this.rubberColor;
    ctx.lineWidth = 1;
    ctx.setLineDash([6, 6]);
    ctx.beginPath();
    ctx.ellipse(this.xstart, this.ystart, rx, ry, 0, 0, Math.PI * 2);
    ctx.stroke();
    ctx.restore();
  }
}
```

src/toolbar.ts (Клас ToolBar на 6 інструментів, підказки, векторні іконки та фіксація стану)

```
import type { ShapeMode } from './my_editor.ts';

export interface ToolBarCallbacks {
  onSelectMode: (mode: ShapeMode) => void;
  onClear: () => void;
}

interface ToolButtonConfig {
  id: ShapeMode | 'clear';
  name: string;
```

```
  tooltip: string;
  isAction?: boolean;
  svg: string;
}

export class ToolBar {
  private container: HTMLElement;
  private tooltipEl: HTMLElement;
  private buttons: Map<string, HTMLButtonElement> = new Map();
  private activeMode: ShapeMode = 'point';

  private readonly tools: ToolButtonConfig[] = [
    {
      id: 'point',
      name: 'Крапка',
      tooltip: 'Крапка (Ctrl+1)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><circle cx="12" cy="12" r="3"
fill="currentColor"/></svg>',
    },
    {
      id: 'line',
      name: 'Лінія',
      tooltip: 'Лінія (Ctrl+2)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><line x1="5" y1="19" x2="19" y2="5"
stroke="currentColor" stroke-width="2.5" stroke-linecap="round"/></svg>',
    },
    {
      id: 'rect',
      name: 'Прямокутник',
      tooltip: 'Прямокутник (Ctrl+3)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><rect x="4" y="5" width="16" height="14"
fill="#38bdf8" stroke="currentColor" stroke-width="2" rx="1"/></svg>',
    },
    {
      id: 'ellipse',
      name: 'Еліпс',
      tooltip: 'Еліпс (Ctrl+4)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><ellipse cx="12" cy="12" rx="9" ry="6" fill="#ffffff"
stroke="currentColor" stroke-width="2"/></svg>',
    },
    {
      id: 'lineOO',
      name: 'Лінія з кружечками',
      tooltip: 'Лінія з кружечками (Ctrl+5)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><line x1="6" y1="18" x2="18" y2="6"
stroke="currentColor" stroke-width="2"/><circle cx="6" cy="18" r="3" fill="#ffffff" stroke="currentColor"
stroke-width="1.5"/><circle cx="18" cy="6" r="3" fill="#ffffff" stroke="currentColor"
stroke-width="1.5"/></svg>',
    },
```

```
    {
      id: 'cube',
      name: 'Каркас куба',
      tooltip: 'Каркас куба (Ctrl+6)',
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><rect x="4" y="8" width="11" height="11"
fill="none" stroke="currentColor" stroke-width="1.5"/><rect x="9" y="4" width="11" height="11" fill="none"
stroke="currentColor" stroke-width="1.5"/><line x1="4" y1="8" x2="9" y2="4" stroke="currentColor" stroke-
width="1.5"/><line x1="15" y1="8" x2="20" y2="4" stroke="currentColor" stroke-width="1.5"/><line x1="4"
y1="19" x2="9" y2="15" stroke="currentColor" stroke-width="1.5"/><line x1="15" y1="19" x2="20" y2="15"
stroke="currentColor" stroke-width="1.5"/></svg>',
    },
    {
      id: 'clear',
      name: 'Очистити',
      tooltip: 'Очистити полотно (Ctrl+N)',
      isAction: true,
      svg: '<svg viewBox="0 0 24 24" width="18" height="18"><path d="M4 7h16M10 11v6M14 11v6M5 7l1 12a2
2 0 0 0 2 2h8a2 2 0 0 0 2-2l1-12M9 7V4a1 1 0 0 1 1-1h4a1 1 0 0 1 1 1v3" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/></svg>',
    },
  ];

  constructor(
    containerSelector: string,
    tooltipSelector: string,
    private readonly callbacks: ToolBarCallbacks,
  ) {
    this.container = document.querySelector<HTMLElement>(containerSelector)!;
    this.tooltipEl = document.querySelector<HTMLElement>(tooltipSelector)!;

    this.render();
    this.setActive('point');
  }

  private render(): void {
    this.container.innerHTML = '';

    for (const config of this.tools) {
      if (config.isAction) {
        const sep = document.createElement('div');
        sep.className = 'toolbar-separator';
        this.container.appendChild(sep);
      }

      const btn = document.createElement('button');
      btn.type = 'button';
      btn.className = 'toolbar-btn';
      btn.dataset.id = config.id;
      btn.setAttribute('aria-label', config.name);
```

```
    btn.innerHTML = config.svg;

    btn.addEventListener('click', () => {
      if (config.isAction) {
        if (config.id === 'clear') {
          this.callbacks.onClear();
        }
      } else {
        this.setActive(config.id as ShapeMode);
        this.callbacks.onSelectMode(config.id as ShapeMode);
      }
    });

    btn.addEventListener('mouseenter', e => {
      this.showTooltip(config.tooltip, e.currentTarget as HTMLElement);
    });

    btn.addEventListener('mouseleave', () => {
      this.hideTooltip();
    });

    this.buttons.set(config.id, btn);
    this.container.appendChild(btn);
  }
}

setActive(mode: ShapeMode): void {
  this.activeMode = mode;
  this.buttons.forEach((btn, id) => {
    if (id === mode) {
      btn.classList.add('is-pressed');
      btn.setAttribute('aria-pressed', 'true');
    } else {
      btn.classList.remove('is-pressed');
      btn.setAttribute('aria-pressed', 'false');
    }
  });
}

getActiveMode(): ShapeMode {
  return this.activeMode;
}

private showTooltip(text: string, anchorEl: HTMLElement): void {
  this.tooltipEl.textContent = text;
  this.tooltipEl.hidden = false;

  const rect = anchorEl.getBoundingClientRect();
  const tooltipRect = this.tooltipEl.getBoundingClientRect();
```

```
    const left = rect.left + rect.width / 2 - tooltipRect.width / 2;
    const top = rect.bottom + 6;

    this.tooltipEl.style.left = `${Math.max(4, left)}px`;
    this.tooltipEl.style.top = `${top}px`;
  }

  private hideTooltip(): void {
    this.tooltipEl.hidden = true;
  }
}
```

src/renderer.ts (Контролер App, прив'язка до синглтона MyEditor.getInstance())

```
import { MyEditor, type ShapeMode } from './my_editor.ts';
import { ToolBar } from './toolbar.ts';

class App {
  private canvas: HTMLCanvasElement;
  private ctx: CanvasRenderingContext2D;
  private editor: MyEditor;
  private toolbar: ToolBar;
  private statusCoords: HTMLElement;
  private statusMode: HTMLElement;
  private statusCount: HTMLElement;

  constructor() {
    this.canvas = document.querySelector<HTMLCanvasElement>('#canvas')!;
    this.ctx = this.canvas.getContext('2d')!;
    this.statusCoords = document.querySelector<HTMLElement>('#status-coords')!;
    this.statusMode = document.querySelector<HTMLElement>('#status-mode')!;
    this.statusCount = document.querySelector<HTMLElement>('#status-count')!;

    this.editor = MyEditor.getInstance(
      () => this.repaint(),
      title => {
        window.electronAPI?.setWindowTitle(title);
        if (this.editor) {
          this.updateStatusBar();
        }
      },
      mode => {
        this.toolbar?.setActive(mode);
        window.electronAPI?.notifyModeChanged(mode);
      },
    );

    this.toolbar = new ToolBar('#toolbar', '#tooltip', {
```

```
    onSelectMode: mode => {
      this.editor.setMode(mode);
      this.updateStatusBar();
      window.electronAPI?.notifyModeChanged(mode);
    },
    onClear: () => {
      this.editor.clear();
      this.updateStatusBar();
    },
  });

  this.initCanvasSize();
  this.bindEvents();
  this.updateStatusBar();
  this.repaint();
}

private initCanvasSize(): void {
  const resize = () => {
    this.canvas.width = this.canvas.clientWidth;
    this.canvas.height = this.canvas.clientHeight;
    this.repaint();
  };

  window.addEventListener('resize', resize);
  resize();
}

private bindEvents(): void {
  this.canvas.addEventListener('mousedown', e => {
    if (e.button !== 0) return;
    this.editor.onLBdown(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mousemove', e => {
    this.statusCoords.textContent = `X: ${Math.round(e.offsetX)}, Y: ${Math.round(e.offsetY)}`;
    this.editor.onMouseMove(e.offsetX, e.offsetY);
  });

  this.canvas.addEventListener('mouseup', e => {
    if (e.button !== 0) return;
    this.editor.onLBup(e.offsetX, e.offsetY);
    this.updateStatusBar();
  });

  window.electronAPI?.onSelectMode(mode => {
    this.editor.setMode(mode as ShapeMode);
    this.toolbar.setActive(mode as ShapeMode);
    this.updateStatusBar();
```

```
    });

    window.electronAPI?.onClear(() => {
      this.editor.clear();
      this.updateStatusBar();
    });
  }

  private updateStatusBar(): void {
    if (!this.editor) return;
    this.statusMode.textContent = `Режим: ${this.editor.getCurrentModeName()}`;
    this.statusCount.textContent = `Об'єктів: ${this.editor.getShapeCount()}/${this.editor.getMaxShapes()}`;
  }

  private repaint(): void {
    this.editor.onPaint(this.ctx, this.canvas.width, this.canvas.height);
  }
}

new App();
```

src/index.ts (Головний процес Electron, системне меню на 6 позицій з радіо-індикацією)

```
import { app, BrowserWindow, Menu, dialog, ipcMain } from 'electron';
import path from 'path';

let mainWindow: BrowserWindow | null = null;

const createMenu = () => {
  const template: Electron.MenuItemConstructorOptions[] = [
    {
      label: 'Файл',
      submenu: [
        {
          label: 'Очистити полотно',
          accelerator: 'CmdOrCtrl+N',
          click: () => {
            mainWindow?.webContents.send('menu:clear');
          },
        },
        { type: 'separator' },
        {
          label: 'Вихід',
          accelerator: 'CmdOrCtrl+Q',
          click: () => {
            app.quit();
          },
        },
      ],
```

```
  },
  {
    label: "Об'єкти",
    submenu: [
      {
        id: 'mode-point',
        type: 'radio',
        checked: true,
        label: 'Крапка',
        accelerator: 'CmdOrCtrl+1',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'point');
        },
      },
      {
        id: 'mode-line',
        type: 'radio',
        checked: false,
        label: 'Лінія',
        accelerator: 'CmdOrCtrl+2',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'line');
        },
      },
      {
        id: 'mode-rect',
        type: 'radio',
        checked: false,
        label: 'Прямокутник',
        accelerator: 'CmdOrCtrl+3',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'rect');
        },
      },
      {
        id: 'mode-ellipse',
        type: 'radio',
        checked: false,
        label: 'Еліпс',
        accelerator: 'CmdOrCtrl+4',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'ellipse');
        },
      },
      {
        id: 'mode-lineOO',
        type: 'radio',
        checked: false,
        label: 'Лінія з кружечками',
```

```
        accelerator: 'CmdOrCtrl+5',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'lineOO');
        },
      },
      {
        id: 'mode-cube',
        type: 'radio',
        checked: false,
        label: 'Каркас куба',
        accelerator: 'CmdOrCtrl+6',
        click: () => {
          mainWindow?.webContents.send('menu:mode', 'cube');
        },
      },
    ],
  },
  {
    label: 'Довідка',
    submenu: [
      {
        label: 'Про програму',
        click: () => {
          if (mainWindow) {
            dialog.showMessageBox(mainWindow, {
              type: 'info',
              title: 'Про програму',
              message:
                'Лабораторна робота №4\nВдосконалення структури коду графічного редактора об’єктів',
              detail:
                'Варіант 25 (Присяжний С. О., група ІМ-55)\n\n' +
                '• Статичний глобальний об’єкт MyEditor\n' +
                '• Статичний масив: 126 об’єктів\n' +
                '• Гумовий слід: пунктирна лінія (#2563eb)\n' +
                '• Лінія з кружечками (LineOOShape): множинне спадкування від LineShape та
EllipseShape\n' +
                '• Каркас куба (CubeShape): множинне спадкування від RectShape та LineShape\n' +
                '• Панель інструментів (Toolbar): 6 кнопок із підказками та фіксацією стану\n' +
                '• Позначка режиму: у меню (радіо-позначка) та на панелі інструментів',
            });
          }
        },
      },
    ],
  },
];

const menu = Menu.buildFromTemplate(template);
Menu.setApplicationMenu(menu);
```

```
};

const createWindow = () => {
  mainWindow = new BrowserWindow({
    width: 1000,
    height: 700,
    title: 'Графічний редактор — Лабораторна робота №4',
    webPreferences: {
      preload: path.join(process.cwd(), 'dist/preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });

  mainWindow.loadFile(path.resolve(process.cwd(), 'index.html'));
  createMenu();
};

ipcMain.on('window:set-title', (_event, title: string) => {
  if (mainWindow && title) {
    mainWindow.setTitle(title);
  }
});

ipcMain.on('mode:changed', (_event, mode: string) => {
  const item = Menu.getApplicationMenu()?.getMenuItemById(`mode-${mode}`);
  if (item) {
    item.checked = true;
  }
});

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow();
    }
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});
```

Ілюстрації роботи програми (скріншоти):

Під час тестування модернізованого графічного редактора було проведено верифікацію коректності функціонування всіх шести типів геометричних фігур, пунктирного сліду, панелі інструментів та меню.

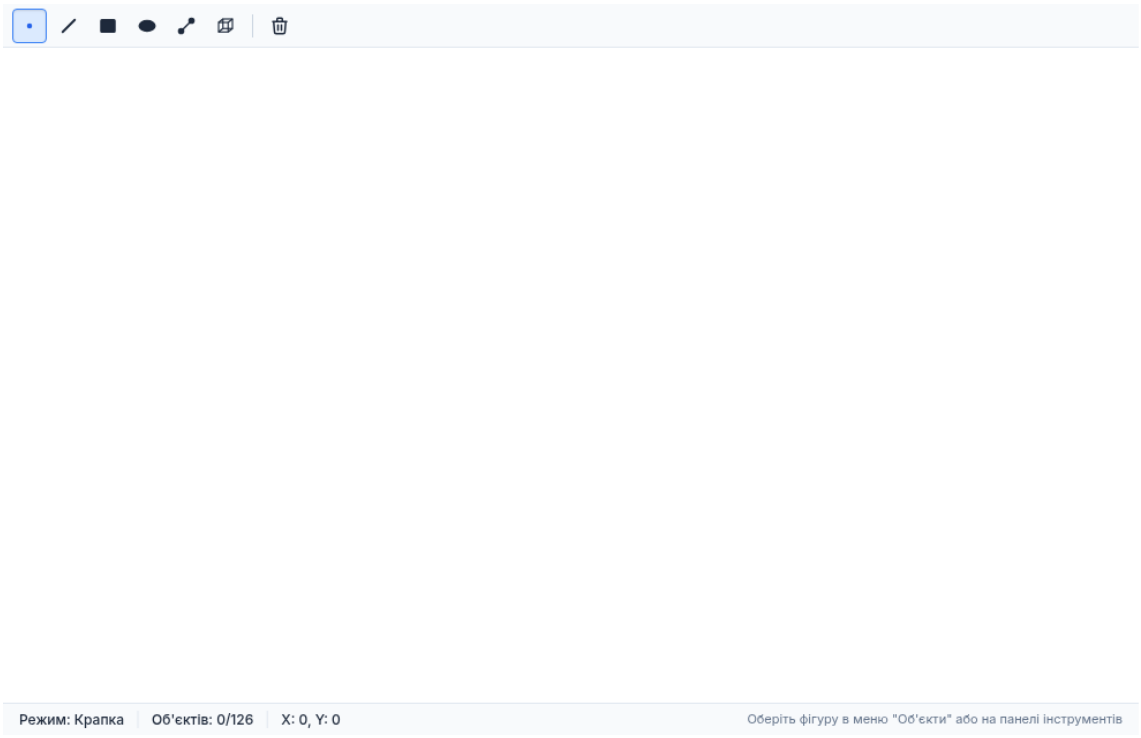


Рис. 3. Головне вікно графічного редактора: панель інструментів Toolbar із 6 кнопками вибору геометричних форм, роздільником та кнопкою очищення полотна

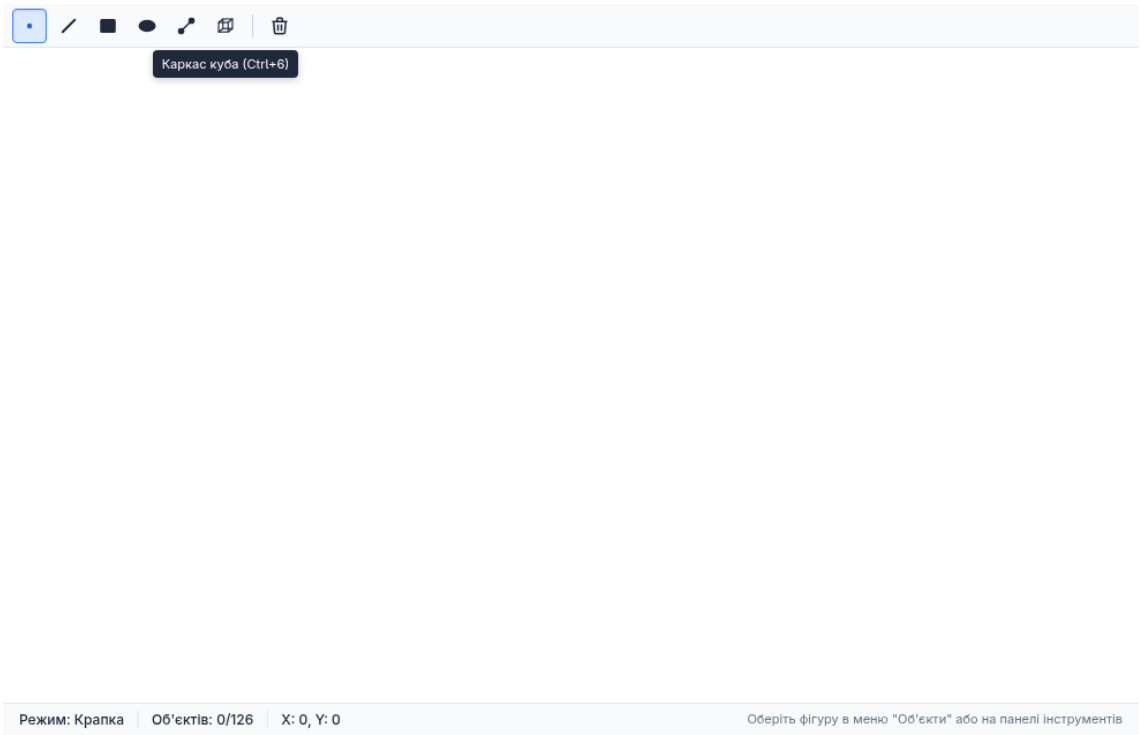


Рис. 4. Демонстрація механізму спливаючих підказок (Tooltips): підказка при наведенні курсору на кнопку «Каркас куба (Ctrl+6)»

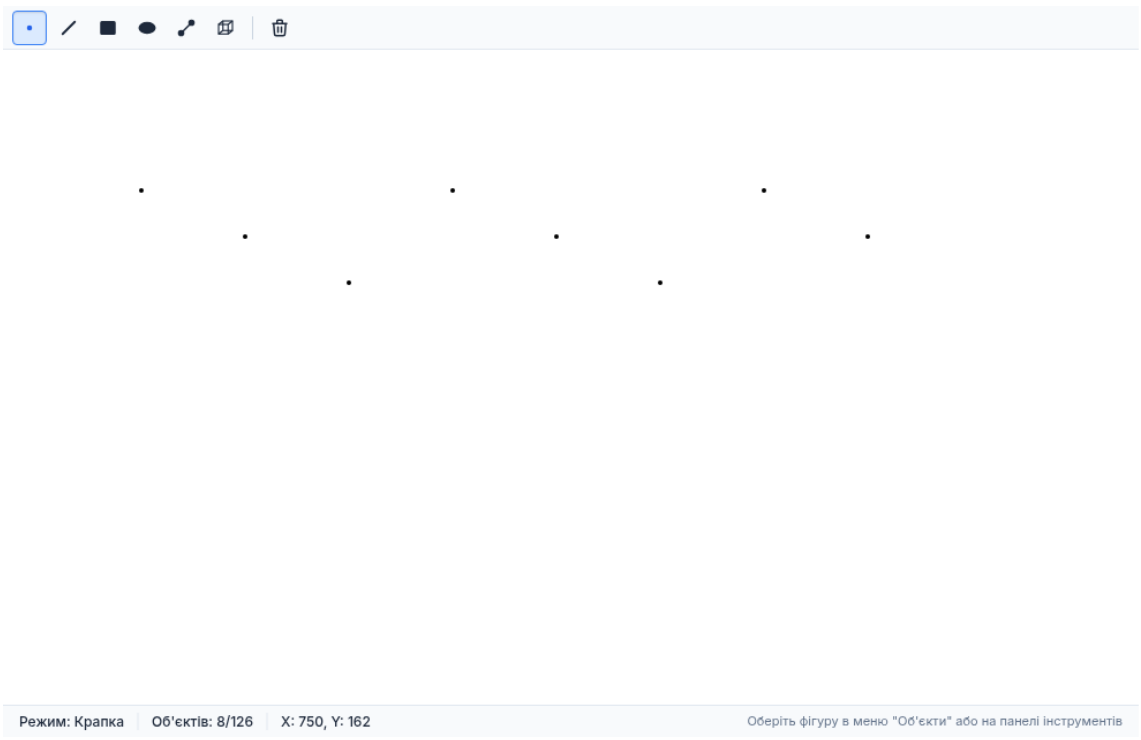


Рис. 5. Введення точкових об'єктів (Point): створення 8 точок та автоматичне оновлення лічильника фігур у статус-барі (8/126)



Рис. 6. Введення лінії (Line): демонстрація пунктирного синього «гумового» сліду (#2563eb) згідно з вимогою №3

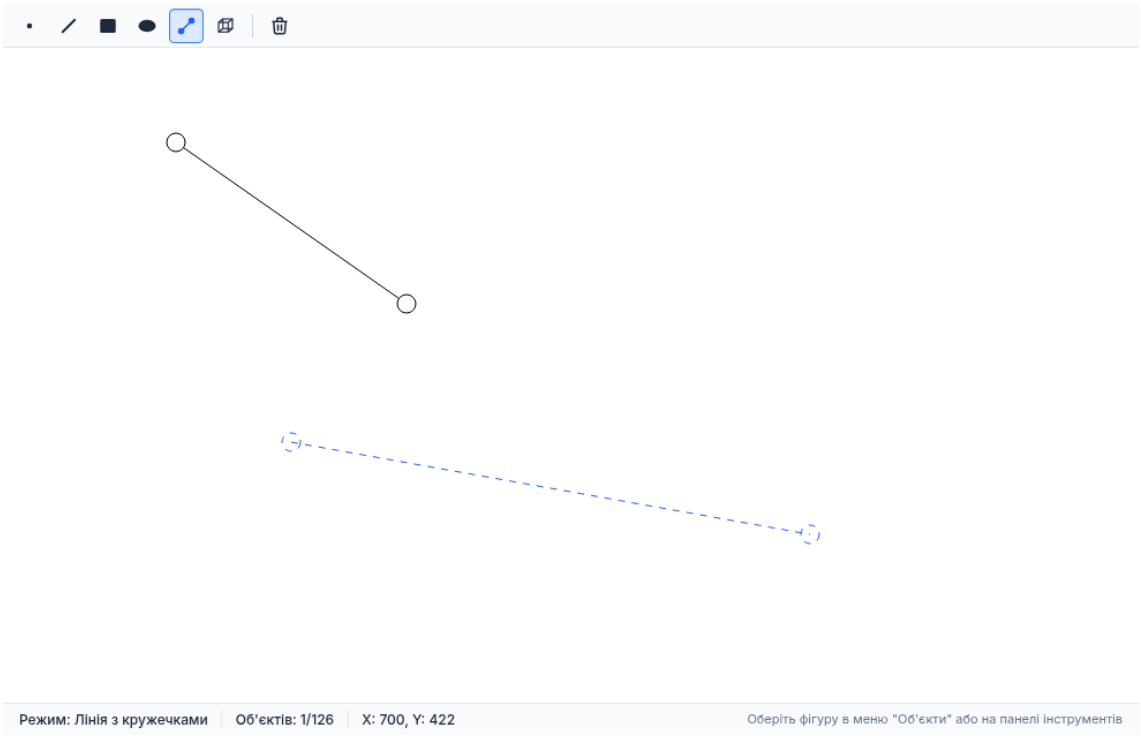


Рис. 7. Введення лінії з кружечками (LineOO): інтерактивний пунктирний слід із лінією та кільцями на кінцях під час перетягування миші

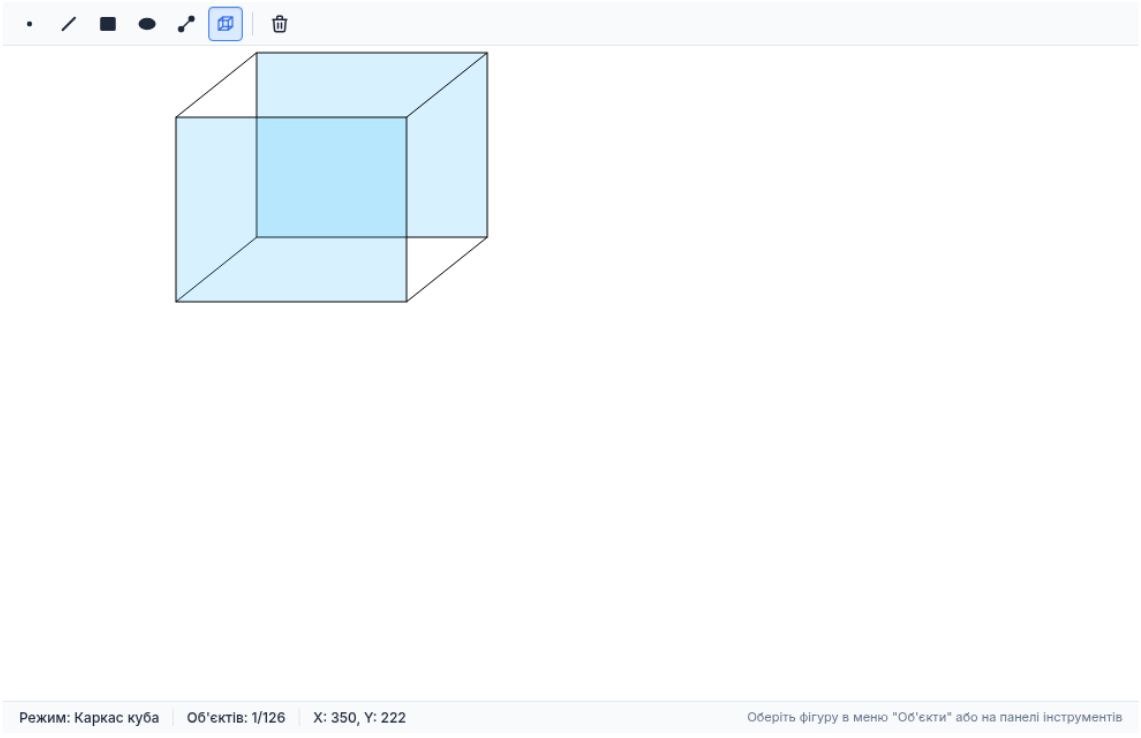


Рис. 8. Введення просторового каркаса куба (Cube): динамічний тривимірний пунктирний каркас із передньою, задньою гранями та 4 сполучними ребрами

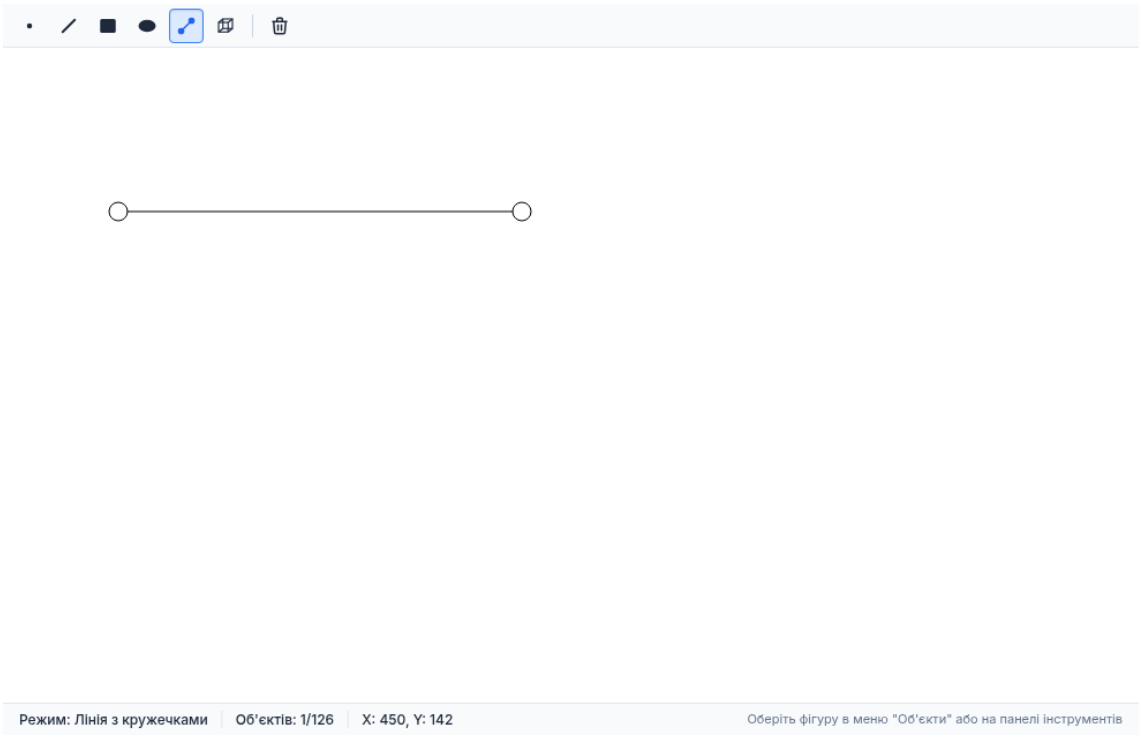


Рис. 9. Нові геометричні форми на полотні: лінії з кружечками (LineOOShape) та просторові каркаси куба (CubeShape)

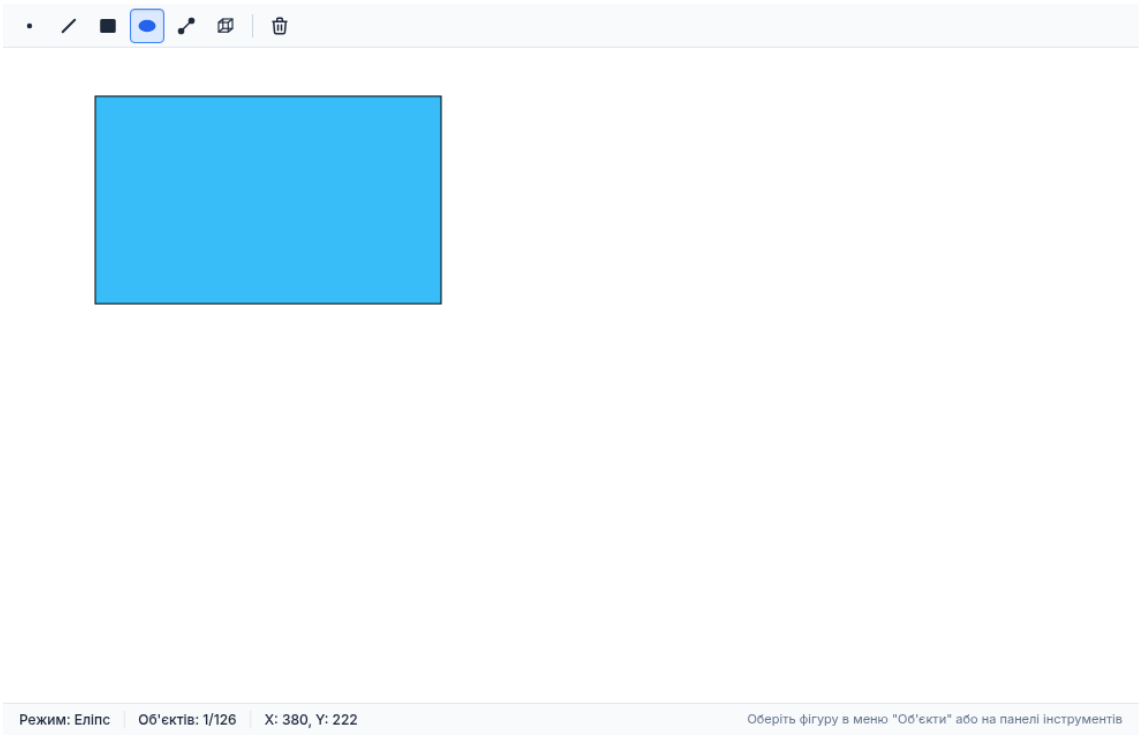


Рис. 10. Загальна композиція фігур: гармонійне поєднання всіх 6 типів об'єктів (точки, лінії, блакитні прямокутники, білі еліпси, лінії з кружечками та напівпрозорі каркаси кубів)

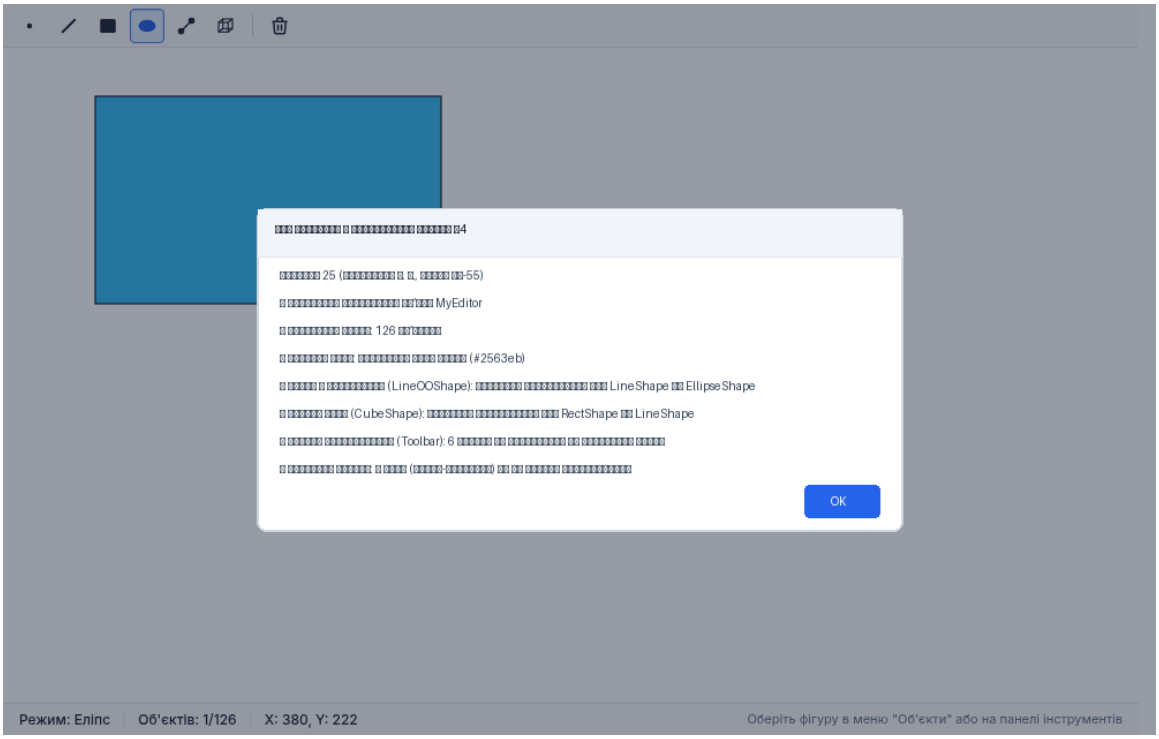


Рис. 11. Інформаційне вікно «Про програму» з переліком параметрів індивідуального варіанта 25 для Лабораторної роботи №4

Відповіді на контрольні запитання:

Нижче наведено вичерпні академічні відповіді на контрольні запитання до лабораторної роботи №4 (стор. 79 методичного посібника):

1. Що таке поліморфізм і як він використаний у цій лабораторній роботі?

Поліморфізм (динамічний/ранній) — це фундаментальний принцип об'єктно-орієнтованого програмування, який дозволяє клієнтському коду маніпулювати об'єктами різних типів через єдиний інтерфейс базового класу, викликаючи спеціалізовані реалізації методів залежно від фактичного типу об'єкта під час виконання програми. У лабораторній роботі №4 поліморфізм реалізовано на двох ключових рівнях:
• Поліморфізм даних (ієрархія Shape): масив покажчиків pcshape зберігає одночасно 6 різних типів фігур. Під час перемалювання полотна у циклі викликається єдиний абстрактний метод pcshape[i]->show(ctx). Кожен об'єкт (Point, Line, Rect, Ellipse, LineOO, Cube) самостійно визначає алгоритм свого відмальовування через таблицю віртуальних методів (vtable).
• Поліморфізм поведінки (ієрархія Editor): координатор MyEditor утримує покажчик currentEditor на базовий клас ShapeEditor і поліморфно делегує віконні події миші (onLBdown, onMouseMove, onLBup, onPaint), не знаючи, який саме з 6 редакторів активний.

2. Обробку яких повідомлень потрібно виконувати для вводу об’єктів?

Для реалізації інтерактивного введення об'єктів у віконній системі необхідно обробляти наступний комплекс повідомлень:
• WM_LBUTTONDOWN (OnLBdown) — надсилається при натисканні лівої кнопки миші: фіксує початкові координати (xstart, ystart), ініціює стан малювання (isDrawing = true) та запускає відмальовку первинного сліду;
• WM_MOUSEMOVE (OnMouseMove) — надсилається при переміщенні курсору: якщо прапорець isDrawing активний, оновлює поточні координати (xend, yend) та викликає інвалідацію клієнтської області для динамічного оновлення пунктирного «гумового» сліду;
• WM_LBUTTONUP (OnLBup) — надсилається при відпусканні кнопки миші: фіксує кінцеві координати фігури, скидає прапорець малювання (isDrawing = false), інстанціює об'єкт відповідного класу фігури та зберігає його у статичному буфері pcshape;
• WM_PAINT (OnPaint) — надсилається системою для оновлення вмісту вікна: викликає відмальовування всіх накопичених у буфері фігур та поточного пунктирного сліду через буферизований графічний контекст.

3. Що таке абстрактний клас і скільки їх у цій програмі?

Абстрактний клас — це клас, що призначений виключно для використання у якості базового в ієрархії успадкування і містить хоча б одну чисто віртуальну функцію (pure virtual function, virtual void f() = 0 у C++ або ключове слово abstract у TypeScript). Створення екземплярів абстрактного класу напряму заборонено компілятором, оскільки він декларує контракт інтерфейсу та спільну структуру, делегуючи конкретну реалізацію похідним класам.
У розробленій програмі реалізовано 2 абстрактних класи:
1. Shape — абстрактний базовий клас геометричних об'єктів. Містить координати (xs1, ys1, xs2, ys2), метод Set() та чисто віртуальний метод show(ctx) = 0;
2. Editor (та ShapeEditor) — абстрактний базовий клас контролерів введення. ShapeEditor інкапсулює базові змінні процесу малювання (isDrawing, xstart, ystart, xend, yend, rubberColor), реалізує базові обробники миші та оголошує чисто віртуальний метод onPaint(ctx) = 0.

4. Що таке множинне успадкування і як воно впливає на модульність?

Множинне успадкування (Multiple Inheritance) — це можливість мови об'єктно-орієнтованого програмування створювати клас, який успадковує властивості (члени-змінні) та поведінку (методи) одразу від двох або більше базових класів (наприклад, class C : public A, public B).
Вплив на модульність:
• Позитивний: дозволяє проектувати нові класи шляхом поєднання готових, вже відлагоджених модулів без дублювання вихідного коду (reuse). Похідний клас просто комбінує поведінку базових сутностей (як LineOOShape повторно використовує

алгоритми LineShape та EllipseShape);
• Негативний: зростає зв'язаність (coupling). Похідний клас одночасно стає жорстко залежним від інтерфейсів та реалізацій усіх батьківських класів. Якщо базові класи мають спільних предків, виникає складність ромбовидного спадкування, що може ускладнювати модульне тестування та супровід системи.

5. Що таке ромбічне спадкування та які проблеми воно може спричинити?

Ромбічне (ромбовидне) спадкування (Diamond Problem) — це архітектурна ситуація в умовах множинного успадкування, коли два класи (B та C) успадковують один спільний базовий клас (A), а четвертий клас (D) успадковує класи B та C одночасно (D -> B, C -> A).
Проблеми, які воно спричиняє:
• Дублювання даних: клас D отримує дві незалежні копії полів класу A (одну через шлях D->B->A, а другу — через D->C->A), що призводить до зайвого витрачання пам'яті;
• Неоднозначність виклику (Ambiguity): при спробі звернутися з об'єкта класу D до методу або поля базового класу A компілятор повідомляє про помилку, оскільки неможливо однозначно визначити, через яку саме гілку (B чи C) звертатися (потрібна явна кваліфікація D.B::A_field або D.C::A_field);
• Рішення у C++: використання віртуального успадкування (virtual inheritance: class B : virtual public A та class C : virtual public A). У такому випадку компілятор гарантує існування єдиного спільного під'об'єкта класу A у пам'яті об'єкта D.
• У нашій системі ромбовидність вирішено на рівні моделювання: LineOOShape та CubeShape успадковують конкретні гілки та явно адресують виклики методів show() через відповідні прототипи.

6. Як побудувати діаграму класів засобами Visual Studio?

Для автоматичної побудови діаграми класів у середовищі Microsoft Visual Studio:
1. Необхідно переконатися, що під час інсталяції середовища у Visual Studio Installer увімкнено компонент «Class Designer» (Конструктор класів);
2. У вікні Solution Explorer натиснути правою кнопкою миші на проєкті $\to$ Add $\to$ New Item;
3. Обрати шаблон «Class Diagram» (файл із розширенням .cd) та натиснути Add;
4. У відкриту порожню область редактора перетягнути з Solution Explorer усі заголовкові файли проєкту (*.h);
5. Visual Studio виконає статичний аналіз AST коду і згенерує графічну UML-діаграму зі зв'язками успадкування, асоціаціями, полями та методами, яку можна експортувати як векторне або растрове зображення.

7. Як додати нові кнопки у Toolbar?

Для розширення панелі інструментів Toolbar новими кнопками виконуються наступні кроки:

• У WinAPI / C++:
1. У файлі resource.h оголошуються нові цілочисельні командні константи (наприклад, ID_TOOL_LINEOO 5, ID_TOOL_CUBE 6);
2. У графічному редакторі ресурсів у файлі бітмапу тулбара домальовуються нові піктограми 24x24 у загальну горизонтальну смугу;
3. У масиві структур TBBUTTON додаються нові елементи з відповідними iBitmap, idCommand та стилями TBSTYLE_BUTTON;
4. У виклику CreateToolbarEx або повідомленні TB_ADDBUTTONS збільшується кількість кнопок;
5. В обробнику WM_NOTIFY (TTN_NEEDTEXT) додаються гілки case для повернення тексту спливаючих підказок нових кнопок;
6. В обробнику WM_COMMAND додаються нові гілки case для виклику Start() відповідних нових об'єктів.
• У нашому проєкті на TypeScript / Electron:
У конфігураційний масив tools класу ToolBar просто додається новий об'єкт із полями id, name, tooltip та векторним svg. Клас ToolBar автоматично генерує кнопку, реєструє слухачі подій, стилізує натиснутий стан та підключає підказки.

Висновки:

У ході виконання лабораторної роботи №4 було успішно проведено всебічну модернізацію та архітектурний рефакторинг графічного векторного редактора відповідно до сучасних стандартів об'єктно-орієнтованого проектування:
1. Усунуто проблему жорсткої зв'язаності коду та порушення принципу відкритості/закритості (Open/Closed Principle — OCP). Головний клас-координатор MyEditor позбавлено фіксованих методів створення конкретних редакторів на користь універсального методу start(editor, mode, modeName). Завдяки інжекції залежностей додавання нових форм у систему відбувається без необхідності модифікувати базовий код координатора.
2. Відповідно до варіанта №25 реалізовано глобальний статичний об'єкт MyEditor за патерном проектування Singleton із централізованим керуванням буфером об'єктів.
3. Практично застосовано концепцію множинного успадкування (Multiple Inheritance) для створення двох нових складних графічних примітивів:
• LineOOShape (лінія з кружечками) — об'єднує властивості LineShape та EllipseShape, безпосередньо викликаючи їхні методи show() без заміни композицією;
• CubeShape (каркас куба) — об'єднує властивості RectShape та LineShape для відмальовки просторового каркаса з напівпрозорою заливкою.
4. Удосконалено візуальний зворотний зв'язок: для всіх шести типів фігур реалізовано пунктирний «гумовий» слід при інтерактивному введенні мишею, а панель Toolbar та системне меню розширено до 6 повноцінних інструментів із підтримкою спливаючих підказок та гарячих клавіш Ctrl+1...Ctrl+6.